CodeAlpha_CreditScoringModel

CodeAlpha Machine Learning Internship — Task 1: Credit Scoring Model

Predicts whether a loan applicant has **Good** or **Bad** credit using financial and demographic features (income, debt, employment status,
credit history, payment history, etc.).

> ℹ️ This is the **compulsory task** for certificate eligibility per the
> internship instructions.

📁 Project Structure

```
CodeAlpha_CreditScoringModel/
├── data/
│   └── credit_data.csv          # dataset (2,000 applicants)
├── src/
│   ├── generate_dataset.py      # creates the dataset
│   └── train_model.py           # preprocessing, training, evaluation
├── outputs/
│   ├── model_comparison.csv     # metrics table for all models
│   ├── confusion_matrix_*.png   # confusion matrix per model
│   ├── roc_curves.png           # ROC curve comparison
```

```
│   ├── feature_importance.png   # top predictive
features
│   └── best_model.pkl           # saved best
pipeline (joblib)
├── main.py                      # runs the full
pipeline end-to-end
├── requirements.txt
└── README.md
```

📊 About the Dataset

No dataset was provided with the task brief, so `generate_dataset.py` creates a **synthetic but realistic** dataset of 2,000 applicants, with financially sensible relationships (e.g. higher debt-to-income ratio, fewer existing loans, longer credit history, and a stable employment status all push an applicant toward "Good" credit), plus injected missing values so the preprocessing step has something real to handle.

Features: Age, AnnualIncome, EmploymentStatus, EmploymentYears, CreditHistoryLength, NumDependents, ExistingLoans, Debt, DebtToIncome, LoanAmount, PaymentHistoryScore
Target: `Creditworthiness` → `Good` / `Bad` (65% / 35% split)

Want to use a real dataset instead? Swap in the [UCI German Credit Data] or a Kaggle credit-scoring dataset — just keep (or remap) the

target
column name to `Creditworthiness` with `Good` / `Bad` values.

🔧 Methodology

1. **Preprocessing** — median imputation for missing numeric values,
   most-frequent imputation for categoricals, one-hot encoding, and
   standard scaling — all inside a single `sklearn` `ColumnTransformer`
   so there's no train/test leakage.
2. **Models trained:**
   - Logistic Regression
   - Decision Tree
   - Random Forest
   - Gradient Boosting (`GradientBoostingClassifier`) — used in place of
     XGBoost because this offline build environment can't install the
     `xgboost` package. It's a drop-in API match: swap in `from xgboost import XGBClassifier` after `pip install xgboost` if
     you want literal XGBoost.
3. **Evaluation** — Accuracy, Precision, Recall, F1, and ROC-AUC on a
   held-out 20% test split (stratified).

📈 Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	**0.840**	0.883	0.869	0.876	**0.916**
Random Forest	0.835	0.873	0.873	0.873	0.910
Gradient Boosting	0.835	0.873	0.873	0.873	0.904
Decision Tree	0.798	0.854	0.831	0.842	0.845

Best model: Logistic Regression (highest ROC-AUC) — automatically selected and saved to `outputs/best_model.pkl`. See `outputs/` for the confusion matrices, ROC curve comparison, and feature-importance chart (top drivers: payment history score, debt-to-income ratio, and credit history length).

▶️ How to Run

```
pip install -r requirements.txt
python main.py
```

This regenerates the dataset, retrains all four models, re-evaluates them, and overwrites everything in `outputs/`.

To use the saved model on new applicants:

```
import joblib
import pandas as pd

model = joblib.load("outputs/best_model.pkl")
new_applicant = pd.DataFrame([{
    "Age": 34, "AnnualIncome": 52000,
"EmploymentStatus": "Employed",
    "EmploymentYears": 6, "CreditHistoryLength":
9, "NumDependents": 1,
    "ExistingLoans": 1, "Debt": 12000,
"DebtToIncome": 0.23,
    "LoanAmount": 8000, "PaymentHistoryScore": 78
}])
prediction = model.predict(new_applicant)       #
1 = Good, 0 = Bad
probability = model.predict_proba(new_applicant) #
[P(Bad), P(Good)]
```

🛠️ Tech Stack

Python · pandas · NumPy · scikit-learn · matplotlib · seaborn · joblib

---

Submitted for the CodeAlpha Machine Learning Internship.